Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 3

Author(s)

Devan Sandhu	sandhd7@mcmaster.ca	sandhd7
Nadeem Mohamed	mohamn84@mcmaster.ca	mohamn84
Billy Wu	wu897@mcmaster.ca	wu897
Nikhil Ranjith	ranjin1@mcmaster.ca	ranjin1
Vivek Patel	patev124@mcmaster.ca	patev124

GitHub URL

https://github.com/sparksavior/2aa4-team1

March 20, 2026

1 Executive Summary

We extended the Catan simulator with 3 design patterns to introduce undo and redo functionality, as well as rule based intelligence, and observer driven architecture. We implemented the Command Pattern by introducing a ReversibleCommand interface and CommandHistory class that manages the undo/redo stacks. Allowed human players to reverse build actions. We applied a Strategy pattern along with Chain of Responsibility to build a rule based system. We evaluated actions using priority weights while constraint rules enforced game obligations using BFS and DFs graph algorithms before value added actions were considered. We also introduced an Observer pattern where Board acts as a Subject and GameStateExplorer as Observer. All 3 patterns were modeled in Papyrus UML diagrams and integrated into existing codebase without major changes to prior classes.

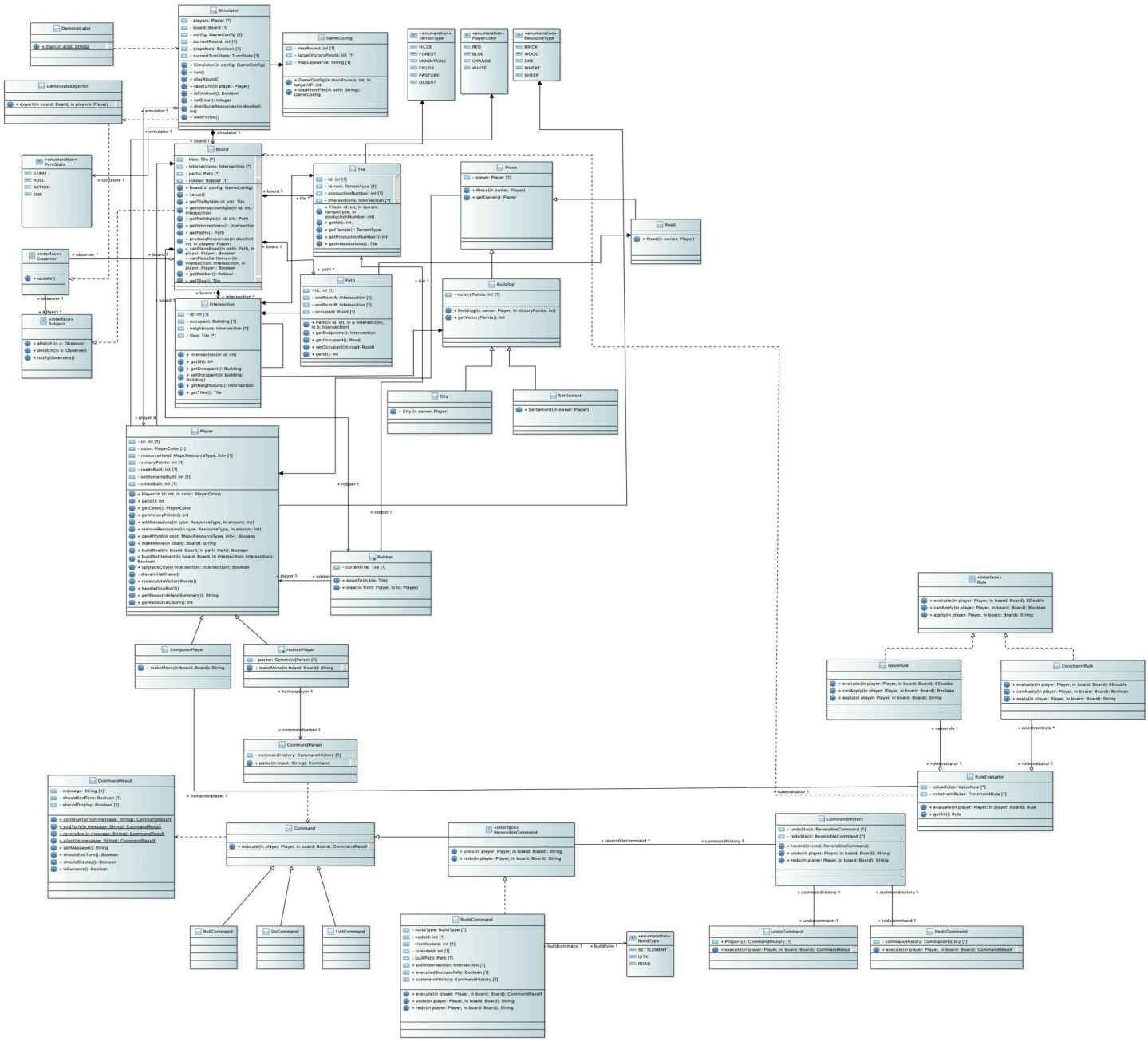
2 Requirements Traceability

Req ID	Status	Implemented in	Design considerations
R3.1	Done	ReversibleCommand.java CommandHistory.java BuildCommand.java UndoCommand.java RedoCommand.java	Used Command Pattern: ReversibleCommand interface extends Command BuildCommand has reversible operations for city, settlement, and road.
R3.2	Done	Rule.java ValueRule.java VictoryPointRule.java BuildRule.java SpendRule.java RuleEvaluator.java ComputerPlayer.java	Strategy Pattern was used here: Rule interface with evaluate(), canApply(), apply() methods. RuleEvaluator selects the highest value rule.
R3.3a	Done	ExcessCardsConstraint.java	Chain of Responsibility used: Used player.getTotalCards() to detect greater than 7 cards. apply() method spends by building city, to settlement, to road .
R3.3b	Done	RoadConnectionConstraint.java	Chain of Responsibility + BFS was used here: Uses board.getRoadsOwnedBy()

			and board.getAdjacentPaths() to detect disconnected road networks via BFS, then finds shortest bridging path (less than or equal to 2 edges) between them.
R3.3c	Done	LongestRoadDefenseConstraint.java	Chain of Responsibility + DFS: Triggers if maxOpponentRoad greater than or equal to myRoad - 1.
R3.3	Done	RuleEvaluator.java	Implemented an evaluate method that goes through a constraint list first. Value rules are only checked when no constraints apply.

3 Design

For a clearer image of the UML, view here or view the papyrus file here.



The UML Diagram for our project.

4 Task 1

Command Pattern

For the implementation of the undo/redo functionality, I decided to use the Command Pattern. The simulator that was already developed contained a representation of player actions in the form of command objects such as BuildCommand, GoCommand, and RollCommand. Since the Command Pattern is a design pattern that encapsulates actions in objects, it is only natural that it supports reversible actions such as undo and redo. Every command object can maintain the information that is required to reverse its own actions.

Explanation and Justification

The Command Pattern encapsulates a request as an object, making it possible to parameterize, store, and execute operations independently of the object that invokes them. Rather than carrying out operations directly in the player or simulator classes, each action is represented by a command class.

In this implementation:

Command represents general player action

ReversibleCommand extends the command interface with undo() and redo() operations

Concrete commands like BuildCommand implement these operations and have the necessary logic to reverse their actions

CommandHistory maintains two Deques that track executed commands allow undo and repo operations

UndoCommand and RedoCommand invoke the corresponding operations in the command history

This approach enhances modularity by separating command execution, command storage, and command invocation. Instead of including the undo operation within the simulator or player classes, each command object holds the responsibility for reversing its own actions. This approach is easier to extend because new commands can provide undo and redo functionality simply by implementing the reversible command interface.

Design and Change

After choosing the Command Pattern, the design became relatively clear. The pattern itself has already specified the roles required for implementing this functionality. These roles include command objects, concrete commands, and an invoker or manager that maintains a list of executed commands. Since the simulator was already using command classes to model actions performed by players, it was easy to incorporate the undo and redo functionality without imposing a new design on the existing system. Most of the design decisions involved determining how commands would retain information about reversing their actions and how the command history would control the undo and redo lists.

Minimal changes were needed to incorporate the pattern into the existing architecture. The basic simulator code and the existing command system were left unchanged. The primary additions involved adding a reversible command interface, implementing the undo and redo functionality in the BuildCommand class, developing a CommandHistory responsible for storing the undo and redo deques,

and defining the UndoCommand and RedoCommand classes. The command parser was modified to handle the new commands as well. Overall, the CommandPattern made it possible to add the undo/redo feature to the existing system with minimal changes, thus facilitating the addition of new features with clean designs.

5 Task 2

Strategy and Chain of Responsibility Pattern

For rule based machine intelligence, Strategy pattern combined with the Chain of Responsibility pattern were chosen. We used Strategy pattern to encapsulate each rule as independent, where each concrete rule class (ExcessCardsConstraint, RoadConnectionConstraint, LongestRoadDefenseConstraint) implemented the same ConstraintRule interface with evaluate(), canApply(), and apply() methods. On the value side, 3 ValueRule classes implement the scoring system, the VictoryPointRule evaluates actions that earn a victory point, such as building settlement or upgrading to city. The BuildRule evaluates building actions that don't directly earn victory points such as building road, and SpendRule evaluates spending cards in such a way that results in less than 5 cards remaining in hand. RuleEvaluator selects the most applicable highest valued rule when no constraints are active. Used Chain of Responsibility Pattern in RuleEvaluator to process constraints sequentially. Constraints checked first in a defined order, and the first constraint that applies is immediately resolved before any value based rules are considered. We chose these patterns because the assignment demanded constraints to be resolved before value added action. Chain of Responsibility enforces this priority ordering by its very own nature, and Strategy pattern allows each rule to be tested, developed, and modified independently without affecting the others.

Explanation and Justification

Strategy pattern encapsulates each decision rule behind a common interface (for example, Rule → ConstraintRule → concrete implementations). So, this decouples decision making logic from RuleEvaluator, which needs to only iterate through a list of rules and pick the best one, never needs to know internal details of how each rule evaluates the board state. Chain of Responsibility ensures RuleEvaluator.evaluate() processes ConstraintRule objects before ValueRule objects. Each constraint in the chain checks whether it applies to current game state and either handles the situation or passes control to the next constraint. This makes it cleaner than a if/else block inside ComputerPlayer.makeMove() because adding or removing a constraint simply requires modifying the list in RuleEvaluator's constructor. No other code requires changing. So, each operates independently, yet they integrate seamlessly through shared ConstraintRule interface.

Once these patterns were identified, structuring the program was very straightforward. The abstract class hierarchy (Rule → ConstraintRule / ValueRule) and RuleEvaluator were already scaffolded. Task was to implement internal logic of each concrete constraint class. The patterns guided the structure in a natural manner because we just needed to answer questions like can constraint apply (canApply), how important is it (evaluate), action to be taken (apply). So, the clear separation meant we could focus on game specific

logic such as BFS for road connectivity, DFS for longest road calculations and we never had to be concerned about how constraints are orchestrated.

Design and Change

The design change was moderate, since the existing codebase did not have rule based system, so Rule, ValueRule, ConstraintRule, RuleEvaluator, and all concrete rule classes were new. But good news being, no existing classes were fundamentally altered. The Board class received read only query helper methods like getRoadsOwnedBy() for example to expose game state to the rules layer. Constraint implementations went through 1 refactoring iteration because initially graph traversal algorithms were directly embedded inside each constraint class, however after careful review among team members, we centralized them into the Board class to keep constraint classes focused on decision logic rather than board level computations.

6 Task 3

The Observer Pattern

Our team chose the Observer pattern for Task 3 because our system has a natural publish/subscribe relationship between a changing game state and the outcomes that react to it. In the current codebase, the real-time visualization was being produced by an explicit class from *Simulator* to *GameStateExporter* after initialization and after each turn, which tightly couples the simulation to a specific output behavior. Observer pattern provides a cleaner separation: the component that owns and changes state (the board) publishes notifications, and the component that reacts to those changes (the exporter) listens and performs the exporting work, which matches the assignments' visualization intent.

Explanation and Justification

Observer works in this project by splitting responsibilities along clear boundaries. *Board* acts as the Subject because it represents the evolving state of roads, intersections/buildings, and robber, and it becomes the central place to manage observers and emit notifications to its subscribers. *GameStateExporter* acts as an Observer because it simply reads the board and exports its content. This improves the design by reducing coupling, improving extensibility, and keeping responsibility aligned with SOLID principles, as *Simulator* orchestrates turns, while exporting logic is encapsulated in the observer.

Design and Change

After deciding on Observer, the main structural thinking was primarily about where the state changes should be considered complete enough to publish. Based on our existing execution flow, we chose to notify at two boundaries from *Simulator*, once after setup so the initial state is visible, once after each player action when the board is in a consistent state. This choice avoids exporting intermediate or partially updated states and preserves the behavior intent of the previous direct-exporting calls.

The amount of change required was minimal and localized. Our team simply added small Observer/Subject interfaces, made *Board* and *GameStateExporter* implement the appropriate roles, and then replaced the direct exporter class inside *Simulator* with *board.notifyObservers()* at the same moments as before. Importantly, the core gameplay mechanics remained the same, as the refactor mainly reorganized how side effects are triggered. This keeps the implementation risk low while still demonstrating the factoring intent.

7 Roles and Responsibilities

The team members contributed equally to the deliverable.